

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA0904

NAME: Rose Mystica. J

COURSE NAME: Programming in Java

REG NO: 192421410

1. Library Book Management using Classes & Encapsulation

PROGRAM:

```
import java.util.*;

class Book {
    private int bookId;
    private String title;
    private String author;
    private double price;
    private boolean issued;

    Book(int id, String t, String a, double p) {
        bookId = id;
        title = t;
        author = a;
        price = p;
        issued = false;
    }

    public int getId() {
        return bookId;
    }

    public void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued");
        }
    }
}
```

```

        } else {
            System.out.println("Book Already Issued");
        }
    }

    public void returnBook() {
        if (issued) {
            issued = false;
            System.out.println("Book Returned");
        } else {
            System.out.println("Book Already Available");
        }
    }

    public void display() {
        System.out.println(bookId + " " + title + " " + author + " " + price + " " + (issued ?
"Issued" : "Available"));
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Book[] b = new Book[2];

        b[0] = new Book(101, "Java", "James", 500);
        b[1] = new Book(102, "Python", "Guido", 450);

        System.out.println("Books Available:");
        for (Book x : b) {
            x.display();
        }
    }
}

```

```

    }
    System.out.print("Enter Book ID to Issue: ");
    int id = sc.nextInt();

    for (Book x : b) {
        if (x.getId() == id) {
            x.issueBook();
        }
    }
    System.out.print("Enter Book ID to Return: ");
    id = sc.nextInt();

    for (Book x : b) {
        if (x.getId() == id) {
            x.returnBook();
        }
    }
}
}

```

OUTPUT:

Output

```

Books Available:
101 Java James 500.0 Available
102 Python Guido 450.0 Available
Enter Book ID to Issue: 101
Book Issued
Enter Book ID to Return: 101
Book Returned

=== Code Execution Successful ===

```

2. Employee Payroll System using Inheritance

PROGRAM:

```
class Employee {
    String name;
    Employee(String name) {
        this.name = name;
    }
    double calculateSalary() {
        return 0;
    }
}

class PermanentEmployee extends Employee {
    double basicSalary;

    PermanentEmployee(String name, double basicSalary) {
        super(name);
        this.basicSalary = basicSalary;
    }
    double calculateSalary() {
        return basicSalary + 5000;
    }
}

class ContractEmployee extends Employee {
    int hoursWorked;
    double ratePerHour;

    ContractEmployee(String name, int hoursWorked, double ratePerHour) {
        super(name);
        this.hoursWorked = hoursWorked;
    }
}
```

```

        this.ratePerHour = ratePerHour;
    }
    double calculateSalary() {
        return hoursWorked * ratePerHour;
    }
}

public class Main {
    public static void main(String[] args) {

        Employee e1 = new PermanentEmployee("Rose", 30000);
        Employee e2 = new ContractEmployee("John", 100, 250);

        System.out.println("Permanent Employee");
        System.out.println("Name : " + e1.name);
        System.out.println("Salary : " + e1.calculateSalary());

        System.out.println();

        System.out.println("Contract Employee");
        System.out.println("Name : " + e2.name);
        System.out.println("Salary : " + e2.calculateSalary());
    }
}

```

OUTPUT:

Output

Permanent Employee

Name : Rose

Salary : 35000.0

Contract Employee

Name : John

Salary : 25000.0

=== Code Execution Successful ===

3. Vehicle Rental System using Runtime Polymorphism

PROGRAM:

```
class Vehicle {
    double calculateRent(int days) {
        return 0;
    }
}

class Car extends Vehicle {
    double calculateRent(int days) {
        return days * 1000;
    }
}

class Bike extends Vehicle {
    double calculateRent(int days) {
        return days * 500;
    }
}

class Bus extends Vehicle {
    double calculateRent(int days) {
```

```
        return days * 2000;
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle[] v = new Vehicle[3];
        v[0] = new Car();
        v[1] = new Bike();
        v[2] = new Bus();

        int days = 3;

        System.out.println("Car Rent = " + v[0].calculateRent(days));
        System.out.println("Bike Rent = " + v[1].calculateRent(days));
        System.out.println("Bus Rent = " + v[2].calculateRent(days));
    }
}
```

OUTPUT:

```
Output
Car Rent = 3000.0
Bike Rent = 1500.0
Bus Rent = 6000.0

=== Code Execution Successful ===
```

4. Bank Account System using Data Hiding

PROGRAM:

```
class BankAccount {  
    private int accountNumber;  
    private String holderName;  
    private double balance;  
  
    BankAccount(int accountNumber, String holderName, double balance) {  
        this.accountNumber = accountNumber;  
        this.holderName = holderName;  
        this.balance = balance;  
    }  
    void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
            System.out.println("Amount Deposited");  
        }  
    }  
    void withdraw(double amount) {  
        if (amount <= balance) {  
            balance -= amount;  
            System.out.println("Amount Withdrawn");  
        } else {  
            System.out.println("Insufficient Balance");  
        }  
    }  
    void displayBalance() {  
        System.out.println("Balance = " + balance);  
    }  
}
```



```

}

public class Main {

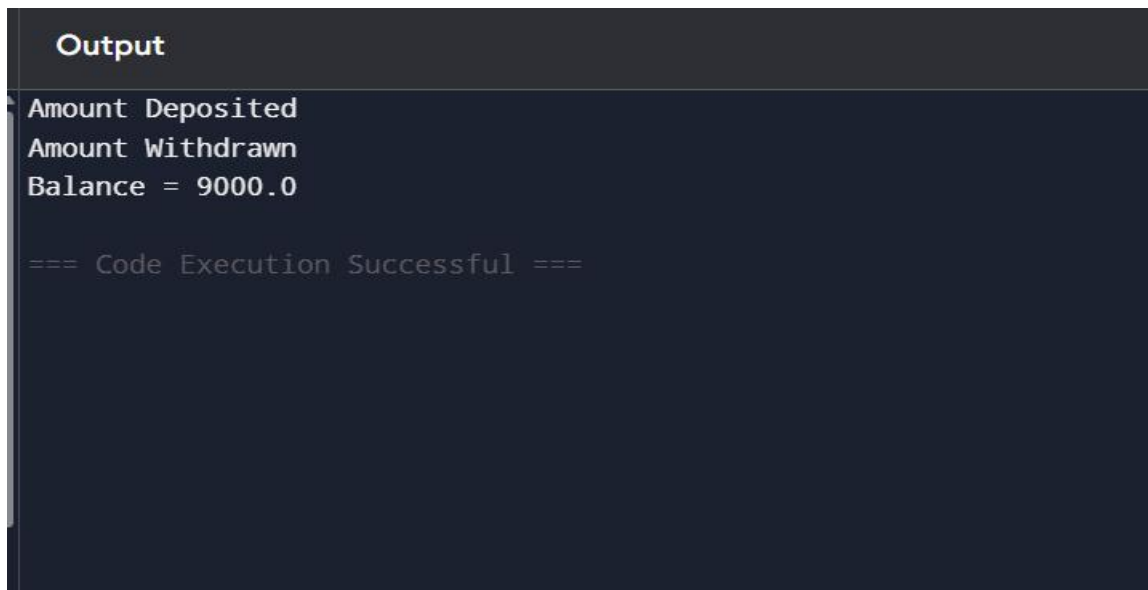
    public static void main(String[] args) {

        BankAccount b = new BankAccount(101, "Rose", 10000);

        b.deposit(2000);
        b.withdraw(3000);
        b.displayBalance();
    }
}

```

OUTPUT:



The screenshot shows a dark-themed output window with the following text:

```

Output
Amount Deposited
Amount Withdrawn
Balance = 9000.0

=== Code Execution Successful ===

```

5. University Admission System using Method Overloading

PROGRAM:

```

class Admission {

    void calculateFee(int fee) {

        System.out.println("Undergraduate Fee = " + fee);
    }
}

```

```
void calculateFee(double fee) {  
    System.out.println("Postgraduate Fee = " + fee);  
}  
void calculateFee(int fee, double scholarship) {  
    System.out.println("Scholarship Fee = " + (fee - scholarship));  
}  
}  
public class Main {  
    public static void main(String[] args) {  
        Admission a = new Admission();  
        a.calculateFee(50000);  
        a.calculateFee(75000.0);  
        .calculateFee(50000, 10000);  
    }  
}
```

OUTPUT:

Output

```
Undergraduate Fee = 50000  
Postgraduate Fee = 75000.0  
Scholarship Fee = 40000.0  
  
=== Code Execution Successful ===
```

6. Shape Area Calculator using Abstract Class & Polymorphism

PROGRAM:

```
abstract class Shape {  
    abstract void area();  
}  
  
class Circle extends Shape {  
    double radius = 7;  
    void area() {  
        System.out.println("Circle Area = " + (3.14 * radius * radius));  
    }  
}  
  
class Rectangle extends Shape {  
    int length = 10;  
    int breadth = 5;  
    void area() {  
        System.out.println("Rectangle Area = " + (length * breadth));  
    }  
}  
  
class Triangle extends Shape {  
    int base = 8;  
    int height = 6;  
    void area() {  
        System.out.println("Triangle Area = " + (0.5 * base * height));  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Shape[] s = new Shape[3];
```

```

s[0] = new Circle();
s[1] = new Rectangle();
s[2] = new Triangle();
for (Shape x : s) {
    x.area();
}
}
}

```

OUTPUT:



The screenshot shows a dark-themed window titled "Output". It contains the following text:

```

Circle Area = 153.86
Rectangle Area = 50
Triangle Area = 24.0

=== Code Execution Successful ===

```

7. Hospital Management System using Interfaces

PROGRAM:

```

interface MedicalRecord {
    void addRecord();
    void displayRecord();
}

class Patient implements MedicalRecord {
    String name = "Rose";
    int age = 20;
}

```

```

    public void addRecord() {
        System.out.println("Patient Record Added");
    }

    public void displayRecord() {
        System.out.println("Patient Name: " + name);
        System.out.println("Age: " + age);
    }
}

class Doctor implements MedicalRecord {
    String name = "John";
    String specialization = "Cardiologist";

    public void addRecord() {
        System.out.println("Doctor Record Added");
    }

    public void displayRecord() {
        System.out.println("Doctor Name: " + name);
        System.out.println("Specialization: " + specialization);
    }
}

public class Main {
    public static void main(String[] args) {

        MedicalRecord p = new Patient();
        MedicalRecord d = new Doctor();

        p.addRecord();
        p.displayRecord();
    }
}

```

```
        d.addRecord();
        d.displayRecord();
    }
}
```

OUTPUT:

```
Output
Patient Record Added
Patient Name: Rose
Age: 20
Doctor Record Added
Doctor Name: John
Specialization: Cardiologist

=== Code Execution Successful ===
```

8. Online Shopping Order System using Packages

PROGRAM:

```
class Product {
    String name;
    double price;

    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}
```

```
class Order {  
    void placeOrder(Product p, int quantity) {  
        double total = p.price * quantity;  
  
        System.out.println("Product: " + p.name);  
        System.out.println("Price: " + p.price);  
        System.out.println("Quantity: " + quantity);  
        System.out.println("Total Amount: " + total);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Product p = new Product("Laptop", 50000);  
        Order o = new Order();  
        o.placeOrder(p, 2);  
    }  
}
```

OUTPUT:

```
Output  
Product: Laptop  
Price: 50000.0  
Quantity: 2  
Total Amount: 100000.0  
  
=== Code Execution Successful ===
```

9. Student Result Analyzer using Arrays of Objects

PROGRAM:

```
class Student {  
    int rollNo;  
    String name;  
    int m1, m2, m3;  
  
    Student(int rollNo, String name, int m1, int m2, int m3) {  
        this.rollNo = rollNo;  
        this.name = name;  
        this.m1 = m1;  
        this.m2 = m2;  
        this.m3 = m3;  
    }  
    int total() {  
        return m1 + m2 + m3;  
    }  
    double average() {  
        return total() / 3.0;  
    }  
    String grade() {  
        if (average() >= 90)  
            return "A";  
        else if (average() >= 75)  
            return "B";  
        else if (average() >= 50)  
            return "C";  
        else  
            return "Fail";  
    }  
}
```



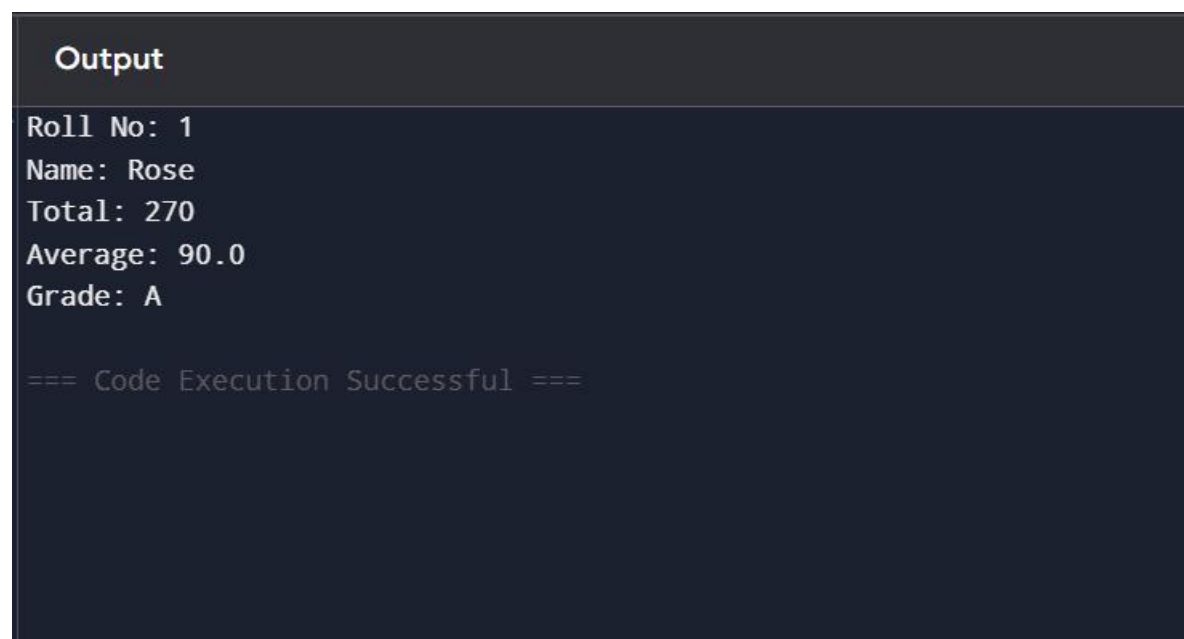
```

    }
}
public class Main {
    public static void main(String[] args) {
        Student s = new Student(1, "Rose", 90, 85, 95);

        System.out.println("Roll No: " + s.rollNo);
        System.out.println("Name: " + s.name);
        System.out.println("Total: " + s.total());
        System.out.println("Average: " + s.average());
        System.out.println("Grade: " + s.grade());
    }
}

```

OUTPUT:



The screenshot shows a dark-themed window titled "Output". It displays the following text:

```

Roll No: 1
Name: Rose
Total: 270
Average: 90.0
Grade: A

=== Code Execution Successful ===

```

10. Smart Home Device Controller using Hierarchical Inheritance

PROGRAM:

```

class Device {
    void turnOn() {

```

```

        System.out.println("Device ON");
    }
    void powerConsumption() {
    }
}
class Light extends Device {
    void powerConsumption() {
        System.out.println("Power Consumption: 20W");
    }
}
class Fan extends Device {
    void powerConsumption() {
        System.out.println("Power Consumption: 75W");
    }
}
class AirConditioner extends Device {
    void powerConsumption() {
        System.out.println("Power Consumption: 1500W");
    }
}
public class Main {
    public static void main(String[] args) {
        Device[] devices = {
            new Light(),
            new Fan(),
            new AirConditioner()
        };
        for (Device d : devices) {
            d.turnOn();

```

```
        d.powerConsumption();  
        System.out.println();  
    }  
}  
}
```

OUTPUT:

```
Output  
Device ON  
Power Consumption: 20W  
  
Device ON  
Power Consumption: 75W  
  
Device ON  
Power Consumption: 1500W  
  
=== Code Execution Successful ===
```